7.0 Test Design Techniques (8 – 11 pm)

105

Test Design Techniques

- 7.1 The Test Development Process
- 7.2 Categories of Test Design Techniques
- 7.3 Specification based or Black Box Techniques
- 7.4 Structure based or White Box Techniques
- 7.5 Experience Based Techniques
- 7.6 Choosing Test Techniques
- 7.7 Summary

106

Terms & Definitions (1)

Dynamic testing
Testing that involves the execution of the software of the component or system.

Test basis
All documents from which the requirements of a component or system can be inferred. The documentation on which the test cases are based.

Test condition
An item or event e.g. a function, transaction, feature, quality attribute, or structural element.of a component or system that could be verified by one or more test cases,

Test design specification
A document specifying the test conditions (coverage items) for a test item, the detailed test approach and identifying the associated high level test cases [after IEEE 829].

107

Terms & Definitions (2)

Test case
Contains the following information:

- for the execution necessary **preconditions**,
- a set of **input values** (one input value per parameter of the test object),
- the amount of **expected results**, as well as
- the **expected postconditions**.

Test cases are developed for a particular objective or test condition, such as to exercise a particular program path or to verify compliance with a specific requirement [after IEEE 610].

Precondition
Environmental and state conditions that must be fulfilled before the test object can be executed with a particular test or test procedure

Postcondition
State conditions (and/or environmental conditions) that must be fulfilled on a test object after the execution of a test or test procedure

108

Terms & Definitions (3)

Test case specification
A document specifying a set of test cases (objective, inputs, test actions, expected results, and execution preconditions) for a test item [after IEEE 829].

Test suite
A set of several test cases for a component or system under test, where the post condition of one test case is often used as the precondition for the next one.

Test procedure specification
A document specifying a sequence of actions for the execution of a test. Also known as test script or manual test script [after IEEE 829].

Test script
Commonly used to refer to a test procedure specification, especially an automated one.

Test execution schedule
A scheme for the execution of test procedures. The test procedures are included in the test execution schedule in their context and in the order in which they are to be executed.

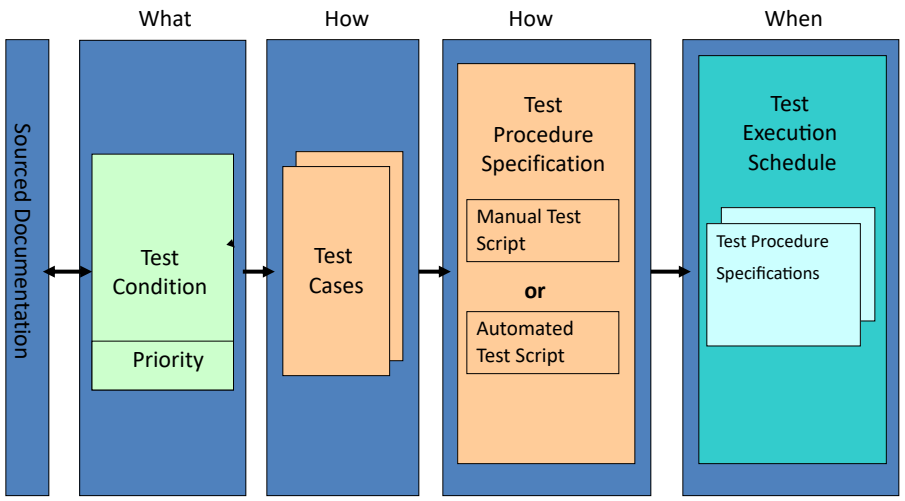
109

7.1 The Test Development Process

- Developing test material can be split into two distinct stages:
  - Defining "what" needs to be tested
  - Defining "how" the system should be tested
- This process can vary from organisation to organisation, can be very formal or very informal with little documentation
- The more formal, the more repeatable the tests, but it does depend on the context of the testing being carried out
- The Test Development Process consists of the following steps:
  - Defining test conditions
  - Specifying test cases
  - Specifying test procedures
  - Developing a Test execution schedule

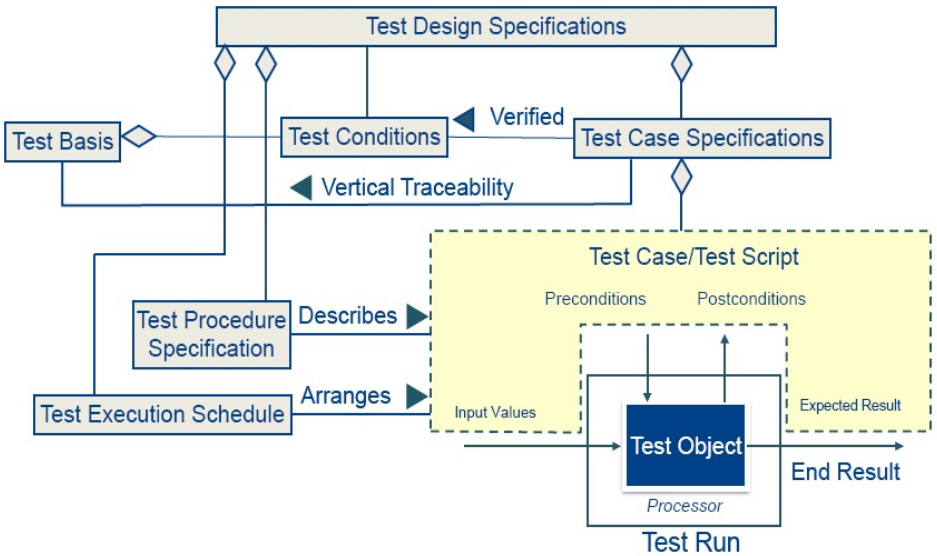
110

Test Conditions, Cases, Procedures and Schedule



111

Dynamic Test: Correlations



112

Dynamic Test – Procedure (1)

- Design of Test to Determine **Test Conditions**
  - Analysis of test based documents, so as to identify, what needs to be tested and also to establish the test conditions
  - Not all Test Conditions are as important as others so each Test Condition is assigned a risk
  - Test conditions must be traceable to specifications or standards, so as to detect the impact on changes of specifications or standards (overlap) on the test (impact analysis).
  - Example:
    - Username field should start with alphabet, and contain at least 2 digits without any special characters.

Dynamic Test – Procedure (2)

- Specifying **Test Cases**
  - Development and detailed description of test cases and test data while using the test design technique
- A Test Case defines how the system should be tested
- They typically contain
  - Input values
  - Execution pre conditions
  - Expected results (output, changes in state etc)
  - Post conditions
  - Cross referenced test conditions
- Remember expected results must be defined before execution
- There can be many Test Cases developed to test a single Test Condition

Expected Results



Testing Pearl of Wisdom

- ***"A necessary part of a test case is a definition of the expected output or result"***

Myers - 1979

Dynamic Test – Procedure (3)

- Specifying the **Test Procedure Specifications**
  - To bring test cases in executable sequence, while considering pre and post conditions of the test cases
- Define the sequence of activities for the execution of a test – in case of using a test execution tool, the test scripts are automated
- Contents of a Test Procedure are:
  - Test procedure specification identifier
  - Purpose
  - Special requirements
  - Procedure steps

Dynamic Test – Procedure (4)

- **Test Execution Schedule**
- The Test Procedure Specifications (i.e. Test Scripts) are subsequently included in a Test Execution Schedule
- This schedule defines the order in which the test scripts are executed, when they are to be carried out and by whom
- The Execution schedule will also need to take account of:
  - Regression Tests
  - Prioritisation
  - And technical and logical dependencies

Static Test vs. Dynamic Test

- Programs are static description of dynamic processes.
- Static tests are checking the test object.
  - Artefacts of the development process, eg. Informal texts, models, formal texts, program code…
- Dynamic tests are checking processes, which result from, interpretation of a description (test object).
- So the test object will be executed on a processor in a dynamic test.
  - Providing of entry values
  - Observing the results.

7.2 Categories of Test Design Techniques

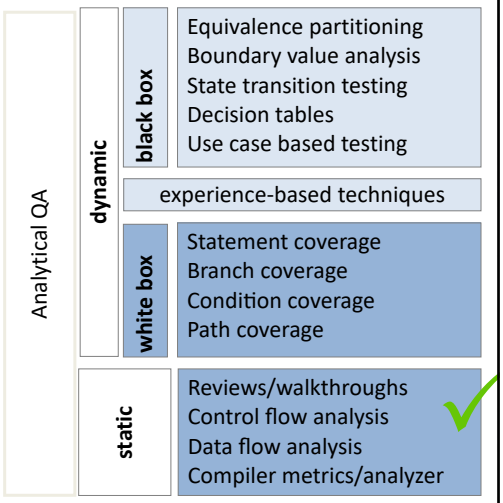
- **Black box and white box testing**

➲ Dynamic testing is divided in two categories/groups

- The grouping is done on the bases of the method to derive test cases

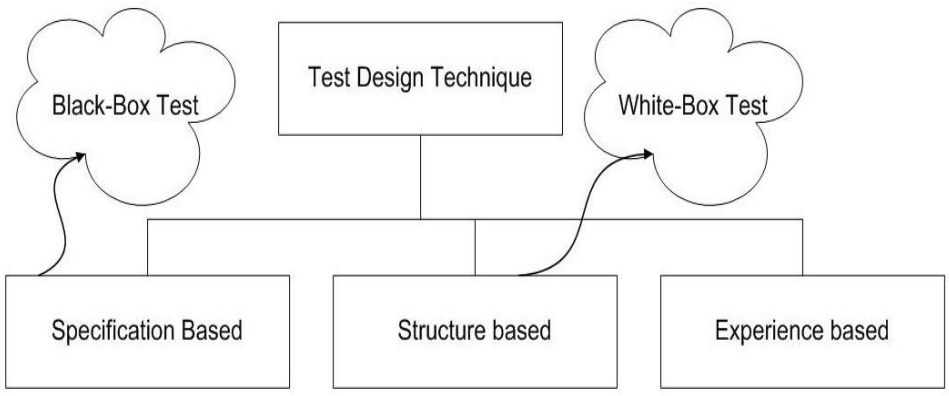
Black Box ⟺ White Box

- Every group has its own set of methods for designing test cases



119

Test Design Techniques



120

Test Design Techniques: Categories

- **Specification-based methods**
  - Test objects have been selected in accordance with the functional software model
  - The coverage of specification can be measured (e.g. which percentage of specification covered by test cases)
- **Structure-based methods**
  - The internal structure of the test object is used to design test cases (code/statements, menus, calls, etc.)
  - The coverage percentage is measured and used as a bases for creating additional test cases
- **Experience-based methods**
  - Knowledge and experience about the test object and its environment are the bases for designing test cases
  - Knowledge and experience about possible weak spots, probable errors and former errors are used to determine test cases

121

Black Box Testing

- **Black-box testing**: *Testing, either functional or non-functional, without reference to the internal structure of the component or system.*



122

Black Box Testing

- testing without knowing the internal workings of the code
- WHAT a system does, rather than HOW it does it
- typically used at System Test phase, although can be useful throughout the test lifecycle
- also known as specification based testing
- Applies for Functional and Non-Functional testing



If Output = Expected result then pass

123

White Box Testing

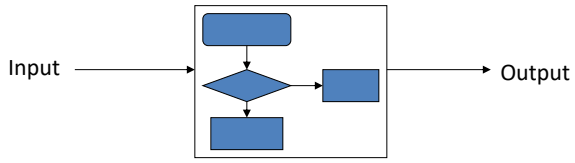
white-box testing: *Testing based on an analysis of the internal structure of the component or system.*



124

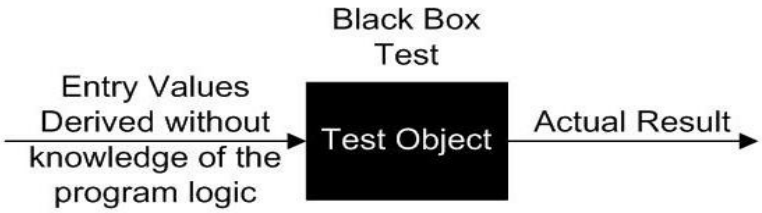
White Box Testing

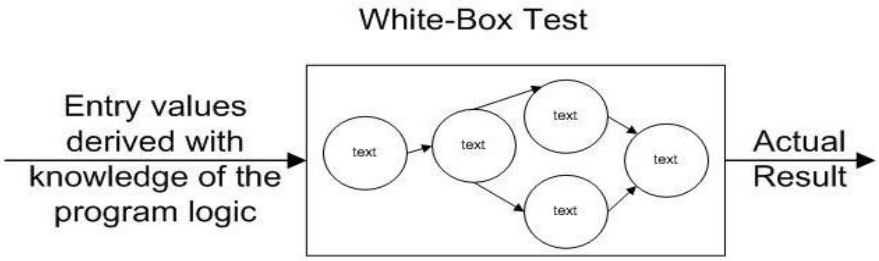
- testing based upon the structure of the code
- typically undertaken at Component and Component Integration Test phases by development teams
- also known as structural or glass box testing or structure based testing



125

Black-Box vs White-Box





126

Experience based Technique

- Uses knowledge and experience of people to derive test cases.
- Knowledge of testers, developers, users and other involved ones about the software, its usage and its environment.
- Knowledge about likely defects and their distribution.

127

Black, White and Experienced based

Black (Specification Based)	•Based on requirements •From the requirements, tests are created •Specification Models can be used for systematic test case design **Techniques** •Equivalence Partitioning •Boundary Value Analysis •Decision Tables •State Transition Testing •Use Case Testing
White (Structure Based)	•Based on code and the design of the system •The tests provide the ability to derive the extent of coverage of the whole application **Techniques** •Statement coverage •Branch Coverage •Decision Coverage
Experience (Black box)	•Based on the knowledge of the tester •Using past experienced use & intuition to "guess" where errors may occur **Techniques** •Error Guessing •Exploratory Testing

128

Use of Techniques and Tools

- techniques provide a systematic method of approaching software testing
- enable tests to be repeatable
- provide a measure of software coverage
- due to the scale and complexity of systems, tools are often used to assist with testing especially white box testing

Test Condition Definition

- Defining Test Condition
  - What is a Test Condition?
  - Where does a Test Condition comes from?

What is a Test Condition?

- Identifies the **"WHAT"** of testing
  - What needs to be tested, AND
  - What are you going to validate about it.
- Includes both ***normal*** and ***error conditions***.
- DO NOT have specific data values assigned to them yet
  - data appears in test cases, the "HOW" of testing

TEST CONDITION ≠ TEST CASES

→ WHAT vs. HOW

Example:
Test the inserting of a record to a table

- Test Conditions identified (among others)

"Validate that you can insert an entry"
"Validate that insertion fails if entry already present"
"Validate that insertion fails if table already full"
"Validate that you can insert an entry to an empty table"

✓ *These are test conditions, NOT test cases because they do not describe the data element being inserted.*
  - *The data is irrelevant at this level, it will appear in the test cases used to cover these test requirements.*

✓ *"Validate you can insert 'John Doe'" is a test case not a test requirements*

Where does Test Conditions come from?

- **Traditional**:

✓ Business Requirements, functionality, internal logic…

➢ Marketing specs, Functional specs, Technical specs

- **Reality**:

✓ "Interview Analysis", non-functional checklist (standards & compliance, Use cases (from scenarios and users), discovery during testing, any other deliverables from previous workbenches (diagrams, modeling, flowcharts, etc.)

133

Guideline to Write Test Conditions

- Use "action" verbs & words
  - "Validate that …"
  - "Check for …"
  - " Test that…"
- Trace them back to the source.

134

In-Class Exercise 1

- Business Requirements:
  - **"ATM must do withdrawals"**
  - **"Withdrawals are between RM10-RM300"**
  - **"Withdrawals are in RM10 multiple"**

- **Derive at least 5 test conditions from the above business requirements**

135

Test Conditions (Answer)

- Validate that the withdrawal >300 and <10 is not allowed.
- Validate that the withdrawal of multiple RM10, between RM10-RM300 can be done.
- Validate that the withdrawal option is offered by the ATM.
- Test that withdrawal of non-multiple RM10 is not allowed.
- Validate that withdrawal is not allowed if the ATM has insufficient money.
- Validate that withdrawal is not allowed is the user has insufficient balance in his account.

136

Test Cases

- "Validate that a withdrawal of a multiple RM10, between RM10-RM300 can be done"

Case #	RM entered	Expected Results	Actual Results	Pass/Fail
WD01	10	RM10 withdrawn		
WD02	20	RM20 withdrawn		
WD03	30	RM30 withdrawn		
:				
WD29	290	RM290 withdrawn		
WD30	300	RM300 withdrawn		

137

7.3 Specification Based Test

- Functional Test
  - Dynamic Test, in which the test cases are being derived under usage of the functional specifications of the test object and the completeness of the testing (coverage) is evaluated according to the functional specifications.
  - The capability of the software product to provide functions which meet stated and implied needs when the software is used under specified conditions [ISO 9126].

138

Specification Based

- Common elements include:
  - Formal or informal models used to specify the problems to be solved, the software or its components.
- Examples include:
  1. Equivalence partitioning
  2. Boundary values analysis
  3. State transition testing
  4. Decision table testing
  5. Use case testing

139

Equivalence Partitioning

- Equivalence class partitioning is what most testers do intuitively: they **divide** the possible values into **classes**. Hereby they look at
  - **input** values of a program (usual use of EC-method)
  - **output** values of a program (rarely used EC-method)
- The range of defined values is grouped into equivalence classes, for which the following rules apply:
  - All values, for which a **common behavior** of the program is **expected**, are grouped together in one equivalence class
  - Equivalence classes may **not overlap** and may **not contain any gaps**
  - Equivalence classes may contain a **range** of values (e.g. $0 < x < 10$) or a **single** value (e.g. x = "Yes")

140

Equivalence Partitioning (cont.)

- The choice of one test value per equivalence provides a useful random sample.
  - If a value of the equivalence class detects a defect, it is hoped for that every other value of the equivalence class will detect this defect.
  - If a value of the equivalence class does not detect a defect, it is hoped for that no other value of the equivalence class will detect a defect.

141

Equivalence Partitioning: valid and invalid

- The equivalence classes of each variable (element) may be divided further
  - **valid EC**: all values within the definition range are combined into one equivalence class, if they are handled identically by the test object
  - **invalid EC**: we distinguish two cases for values outside of the definition range:
    - Values with a correct format but outside of the value range can be combined into one or more equivalence classes
    - Values with a wrong format generally build a separate EC
- Tests are performed using a **single representative** from each EC
  - For every other value from the EC the same behavior as for the chosen value is expected

142

EP: Example

- Equivalence classes are chosen for **valid** and **invalid** inputs
  - If a value x is defined as **0 ≤ x ≤ 100**, then we can initially identify three equivalence classes:
    1. x < 0 (invalid input values)
    2. 0 ≤ x ≤ 100 (valid input values)
    3. x > 100 (invalid input values)
  - Further invalid EC can be defined, containing, but not limited to:

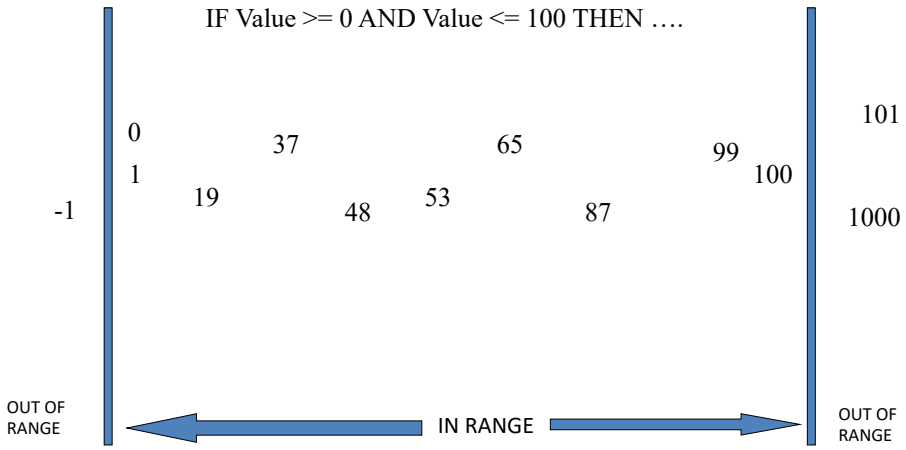
    non-numerical inputs,
    numbers too big or too small,
    non-supported format for numbers



143

144

Equivalence Partitioning



145

Procedure (cont.)

- Equivalence partitioning for every further limitation:
  - If a limitation specifies the area of validity:
    - One valid and two invalid equivalence classes.
  - If a limitation specifies a minimal and a maximal amount of values:
    - One valid and two invalid equivalence classes.
  - If a limitation specifies an amount of values, which possibly will be treated differently:
    - For each of this amount an own valid equivalence class an in addition to that all together one invalid equivalence class.
  - If a limitation specifies a situation, which must be met mandatory:
    - One valid and one invalid equivalence classes.

146

Equivalence Partitioning Example (3)

- If a limitation specifies the area of validity: One valid and two invalid equivalence classes.
  - In the specification of the test object, it is defined that integer input values from 1 to 100 are possible.
  - Area of validity: 1 <= x <= 100
  - Valid class: 1 <= x <= 100
  - Invalid classes: x <1 as well as x >100

147

Equivalence Partitioning Example (4)

- If a limitation specifies a minimal and a maximal amount of values: One valid and two invalid equivalence classes.
  - Specification: A member of the sports club has to register himself with at least one type of sport. Every member can only be registered actively in a maximum of three types of sport.
  - Valid class: 1 <=x <=3 (1 to 3 sports)
  - Invalid classes:
    - x = 0 (no allocation to a sport)
    - x > 3 (allocated to more than 3 sports)

148

Equivalence Partitioning Example (5)

- If a limitation specifies an amount of values, which possibly will be treated differently: For each of this amount an own valid equivalence class an in addition to that all together one invalid equivalence class.
  - Specification: The sports club offers following sports: Football, Hockey, Basketball and Volleyball.
  - Valid classes:
    - Football
    - Hockey
    - Basketball
    - Volleyball
  - Invalid classes: anything else e.g.: Badminton

149

Equivalence Partitioning Example (6)

- If a limitation specifies a situation, which must be met mandatory: One valid and one invalid equivalence classes.
  - Specification: Every member of the sports club received a unique member ID. It starts with the first letter of the family name of the member.
  - Valid class:
    - First digit is a letter.
  - Invalid class:
    - First digit is not a letter.

150

Equivalence Partitioning

- EP can help reduce the number of tests from a list of all possible inputs to a minimum set that would still test each partition
- If the tester chooses the right partitions, the testing will be accurate and efficient
- If the tester mistakenly thinks of two partitions as equivalent and they are not, a test situation will be missed
- Or on the other hand, if the tester thinks two objects are different and they are not, the tests will be redundant
- EP can be used for all Levels of Testing
- EP is used to achieve good input and output coverage, knowing exhaustive testing is often impossible
- It can be applied to human input, input via interfaces to a system, or interface parameters in integration testing

In-Class Exercise 2

- The percentage value will now be displayed in a **bar chart**.
  The following additional requirements apply (both values included):
  - values between 0 and 15: grey bar,
  - values between 16 and 50: green bar,
  - values between 51 and 85: yellow bar,
  - values between 86 and 100: red bar.

EP – picking representatives

- On a last step, one representative of each EC is determined as well as the result to be expected for it

Variable	Equivalence class	Representatives
percentage value (valid)	EC_1: $0 \le x \le 15$	+10
	EC_2: $16 \le x \le 50$	+20
	EC_3: $51 \le x \le 85$	+80
	EC_4: $86 \le x \le 100$	+90
percentage value (invalid)	EC_5: $x < 0$	–10
	EC_6: $x > 100$	+200
	EC_7: x no integer	1,5
	EC_8: x non numeric	fred

153

In-Class Exercise 3

- Consider the following scenario.
  - If you take a train before 9:00 am or in the evening after 5:00 pm until 6.30 pm (the rush hour), you must pay full fare. A saver ticket is available for trains between 9.00 am and 5.00 pm, and after 6.30 pm. The first train departs from KL Sentral at 5 am and the last train is at midnight.
- What are the partitions to test the train times for ticket types? Derive the test cases for the partitions.
- Which are valid partitions and which are invalid partitions?

154

Equivalence Partitioning : Defect Masking

- Certain defects might possibly remain undetected if different invalid equivalence classes are combined in the same test case!

- Example
  - Input Value 1 <= value <= 99; colour IN (red, green, yellow)
- Equivalence Classes
  - value_VP1: 1 <= value <= 99
  - value_IP1: value < 1
  - value_IP2: value > 99
  - colour_VP1: colour IN (red, green, yellow) e.g. red
  - colour_IP1: NOT colour IN (red, green, yellow) e.g blue
- Test data value=0, colour=blue -> value_IP1 and colour_IP1
- ⇒ Any defective treatment of colour=blue maybe remain undetected

155

EP: Analyzing the specification

- A piece of code computes the price of a product based on its value, a discount in % and shipping costs (RM6, RM9 or RM12, depending on shipping mode)
- **Requirements:**
  - ➲Value of goods is given as a positive number with 2 decimal places
  - ➲Discount is a percentage value without decimal places between 0% and 100%
  - ➲Shipping costs can only be 6, 9 or 12

➲Derive the EP class and representative values for each class.

156

Variable	Equivalence Class	Status	Representative
Value of goods	EC_{11}: x >= 0.00	valid	1000.00
	EC_{12}: x < 0.00	invalid	–1000.00
	EC_{13}: x non-numerical value	invalid	fred
Discount	EC_{21}: $0\% \leq x \leq 100\%$	valid	10%
	EC_{22}: x < 0%	invalid	–10%
	EC_{23}: x > 100%	invalid	200%
	EC_{24}: x non-numerical value	invalid	fred
Shipping costs	EC_{31}: x = 6	valid	6
	EC_{32}: x = 9	valid	9
	EC_{33}: x = 12	valid	12
	EC_{34}: $x \neq \{6, 9, 12\}$	invalid	4
	EC_{35}: x non-numerical value	invalid	fred

157

- **All test cases:**
  - 10 test cases are derived: 3 positive (valid values) and 7 negative (invalid values) test cases:

Variable	Status	Representative	T01	T02	T03	T04	T05	T06	T07	T08	T09	T10
Value of goods	valid	1000	X	X	X	X	X			X	X	X
	invalid	–1000.00						X				
	invalid	fred							X			
Discount	valid	10%	X	X	X	X	X	X	X			
	invalid	–10%								X		
	invalid	200%									X	
	invalid	fred										X
Shipping costs	valid	6	X					X	X	X	X	X
	valid	9		X								
	valid	12			X							
	invalid	4				X						
	invalid	fred					X					
Expected Output												
Price			906									
Error Message						"Invalid Shipping costs"	"Invalid Shipping costs"					
Actual Output												
Price												
Error Message												
Pass / Fail												

158

- **Test cases for valid EC:**
  - Valid equivalence classes provide the following combinations or test cases: T01, T02 and T03

Variable	Equivalence Class	Status	Representative	T01	T02	T03
Value of goods	EC_{11}: x >= 0	valid	1000.00			
	EC_{12}: x < 0	invalid	–1000.00			
	EC_{13}: x non-numerical value	invalid	fred			
Discount	EC_{21}: $0\% \leq x \leq 100\%$	valid	10%			
	EC_{22}: x < 0%	invalid	–10%			
	EC_{23}: x > 100%	invalid	200%			
	EC_{24}: x non-numerical value	invalid	fred			
Shipping costs	EC_{31}: x = 6	valid	6			
	EC_{32}: x = 9	valid	9			
	EC_{33}: x = 12	valid	12			
	EC_{34}: $x \neq \{6, 9, 12\}$	invalid	4			
	EC_{35}: x non-numerical value	invalid	fred			

159

- **Test cases for invalid EC:**
  - The following test cases were created using the invalid EC, each in combination with valid ECs of other elements:

Variable	Equivalence Class	Status	Representative	T04	T05	T06	T07	T08	T09	T10
Value of goods	EC_{11}: x >= 0	valid	1000,00							
	EC_{12}: x < 0	invalid	–1000,00							
	EC_{13}: x non-numerical value	invalid	fred							
Discount	EC_{21}: $0\% \leq x \leq 100\%$	valid	10%							
	EC_{22}: x < 0%	invalid	–10%							
	EC_{23}: x > 100%	invalid	200%							
	EC_{24}: x non-numerical value	invalid	fred							
Shipping costs	EC_{31}: x = 6	valid	6							
	EC_{32}: x = 9	valid	9							
	EC_{33}: x = 12	valid	12							
	EC_{34}: $x \neq \{6, 9, 12\}$	invalid	4							
	EC_{35}: x non-numerical value	invalid	fred							

160

Equivalence Partitioning – in general /1

- Partitioning
  - –The quality of the test depends on precisely segmented variables/elements in equivalence classes
  - –EC that were not identified hold the risk of overlooking possible defects, since the representatives used did not cover all possibilities
- Test cases
  - –Equivalence class method provides test cases for which a representative still has to be chosen
  - –Test data combinations are selected by defining the representative or representatives of each equivalence class

161

EP – In general (2)

- choosing representatives
  - –**any value** within the EC can be a **representative**. Optimal are:
    - typical values (used often)
    - problem values (suspected failures)
    - boundary values (on the edge of the EC)

➲Representatives of **valid EC** may be **combined**

➲Representatives of **invalid EC** may **not be combined**

➲Representatives of **invalid EC** may only be **combined with valid** representatives of **other EC**

➲For test cases, representatives of invalid EC should be combined with always the same values of other valid EC (standard combinations)

➲Choosing representatives implies that the function within the program uses compare operations

162

EP– coverage

- Equivalence class coverage can be used as exit criteria to end testing activities

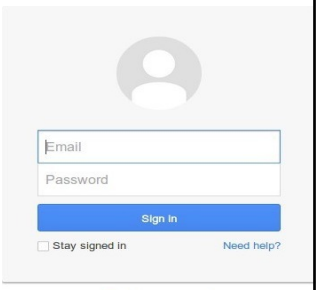
$$EC\text{-}Coverage = \frac{Number\ of\ EC\ tested}{Number\ of\ EC\ defined} * 100\%$$

- Example:
  - If 18 equivalence classes have been determined from the statements or specifications of the input date, and if only 15 of these have been tested in test cases, an equivalence class coverage of 83 % has been reached.
  - Equivalence class coverage = (15 / 18) * 100% = 83,33 %

163

In-Class Exercise 4

- The requirements are as follows:
  - Username should contain letter, number and period.
  - Username should not be left blank.
  - Username should not be more than 40 characters.
  - Username should not start with or contain any symbols.
  - Password should be at least 6 characters.
  - Password should contain combination of letter, numbers and symbols.
  - Password should not contain spaces and period.
  - Password should not be more than 40 characters.



Gmail Login Screen

i. Write test conditions needed to test the site.

ii. Use equivalence class partitioning to create tests cases for respective test conditions written.

164